

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a86483e1c59de33f8.jsonl`  
- SHA-256: `1e01d9245698634ae20118e2e996736145f7ed8a3ea9d0f5cf4f84a0d8c08162`  
- Source modified: `2026-06-10T06:12:29+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_2a64bc88-2a8`  
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`
```

Transcript

1. user (2026-06-10T06:10:58.748Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

ROLE: adversarial verifier. An auditor ("reliability") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.

FINDING: {"title":"WSL detector default timeout_sec=0 means a hung GPU/conda call blocks the synchronous request forever","severity":"high","area":"reliability / WSL seam","file":"backend/pipeline/detectors/wasb_runner.py:40 & :54 (timeout_sec default 0); tracknet_runner.py:41 & :55; passed as `timeout=(self.cfg.timeout\_sec or None)` at wasb_runner.py:67 / tracknet_runner.py:102","evidence":"WasbConfig/TrackNetConfig default timeout_sec=0, and `from\_indexing\_cfg` defaults the JSON key to 0; config.json:54 ships 0. `\_wsl\_bash` converts `0 or None` -> None, i.e. subprocess.run with no timeout. A WSL GPU driver hang, a conda activation prompt, or a stuck predict.py blocks the worker thread indefinitely.","why_it_matters":"/api/process runs inline (DEFERRED #16 problem statement), so one hung WSL call ties up a request thread with no recovery ? the prompt called this exact timeout_sec=0 hang out. The deferral list names cancellation/timeout as a future async concern but does not flag that the shipped default is already infinite.","why":"placeholder"} Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:11:07.377Z)

```
1      """"WASB shuttle-trajectory runner (WSL2 subprocess adapter).  
2
```

```

3      Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4      1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5      2. copies wasb_infer.py into the WASB repo's src/ dir,
6      3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7      4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9      So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10     with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11     """
12     import os
13     import shlex
14     import logging
15     import subprocess
16     from dataclasses import dataclass
17     from typing import Dict, Any, Optional, Tuple
18
19     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20     # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
21     TrackNetRunner.
22     from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
23     from backend.pipeline.detectors.base import DetectorRunner
24
25     logger = logging.getLogger(__name__)
26
27     # Windows path to the inference wrapper we stage into WSL.
28     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
29
30     @dataclass
31     class WasbConfig:
32         """Resolved settings for a WASB WSL run (built from config.json ->
33         indexing.wasb)."""
34         distro: str = "Ubuntu"
35         repo_dir: str = "~/models/WASB-SBDT"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "wasb"
38         weights_path: str = "~/models/WASB-
39         SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
40         sport: str = "badminton"
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 0 # 0 = no timeout
43
44     @classmethod
45     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
46         w = (idx_cfg or {}).get("wasb", {}) or {}
47         return cls(
48             distro=w.get("wsl_distro", "Ubuntu"),
49             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
50             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
51             conda_env=w.get("conda_env", "wasb"),
52             weights_path=w.get("weights_path",
53                               "~/models/WASB-
54                               SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
55             sport=w.get("sport", "badminton"),
56             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(w.get("timeout_sec", 0)),
58         )
59
60     class WasbRunner(DetectorRunner): # DetectorRunner ABC (finish item 6 / new item 1)
61         def __init__(self, cfg: WasbConfig):
62             self.cfg = cfg
63
64         def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
65             full = f"source {self.cfg.conda_sh} && {inner_cmd}"

```

```

64         argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
65         logger.debug("WSL exec: %s", full)
66         return subprocess.run(argv, capture_output=True, text=True,
67                               timeout=(self.cfg.timeout_sec or None))
68
69     def healthcheck(self) -> Tuple[bool, str]:
70         """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
71         cmd = (f"conda activate {self.cfg.conda_env} && "
72               f"python -c \"import torch; print('torch', torch.__version__, "
73               f"'cuda', torch.cuda.is_available())\"")
74         try:
75             res = self._wsl_bash(cmd)
76         except FileNotFoundError:
77             return False, "`wsl` not found ? Windows + WSL2 required."
78         except subprocess.TimeoutExpired:
79             return False, "WSL healthcheck timed out."
80         out = (res.stdout or "").strip()
81         if res.returncode != 0:
82             return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
83         return True, out
84
85     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
86         src_mnt = to_wsl_mnt_path(video_win_path)
87         dest = f"{self.cfg.wsl_stage_dir}/{base}"
88         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)} "
89         {shlex.quote(dest)} && echo OK"
90         try:
91             res = self._wsl_bash(cmd)
92         except subprocess.TimeoutExpired:
93             logger.error("Staging video into WSL timed out.")
94             return None
95         if res.returncode != 0 or "OK" not in (res.stdout or ""):
96             logger.error("Failed to stage video into WSL: %s", (res.stderr or
97 res.stdout).strip())
98             return None
99         return dest
100
101     def _stage_infer_script(self) -> bool:
102         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
103         src_mnt = to_wsl_mnt_path(_INFER_WIN)
104         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
105 OK"
106         res = self._wsl_bash(cmd)
107         return res.returncode == 0 and "OK" in (res.stdout or "")
108
109     def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
110         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or
111 None."""
112         os.makedirs(output_win_dir, exist_ok=True)
113         # Normalize for cross-platform basename (tests use Windows paths on Linux).
114         video_win_path = str(video_win_path).replace("\\", "/")
115         base = os.path.basename(video_win_path)
116         stem = os.path.splitext(base)[0]
117         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
118         expected_csv_win = os.path.join(output_win_dir, f"{stem}_wasb.csv")
119
120         if not self._stage_infer_script():
121             logger.error("Could not stage wasb_infer.py into WSL.")
122             return None
123         staged_video = self._stage_video(video_win_path, base)
124         if staged_video is None:
125             return None
126         return None
127         frames_dir = f"{self.cfg.wsl_stage_dir}/{stem}_frames"
128
129         cmd = (

```

```

125         f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
126         f"python wasb_infer.py "
127         f"--video {shlex.quote(staged_video)} "
128         f"--frames_out_dir {shlex.quote(frames_dir)} "
129         f"--weights {shlex.quote(self.cfg.weights_path)} "
130         f"--sport {shlex.quote(self.cfg.sport)} "
131         f"--out {shlex.quote(wsl_out_dir + '/' + stem + '_wasb.csv')}}"
132     )
133     logger.info("Running WASB inference on %s ...", base)
134     try:
135         res = self._wsl_bash(cmd)
136     except subprocess.TimeoutExpired:
137         logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
138         return None
139     if res.returncode != 0:
140         logger.error("WASB inference failed:\n%s", (res.stderr or
141 res.stdout)[-2000:])
142         return None
143     if not os.path.exists(expected_csv_win):
144         logger.error("WASB finished but expected CSV not found at %s",
145 expected_csv_win)
146         return None
147     logger.info("WASB trajectory CSV: %s", expected_csv_win)
148     return expected_csv_win
149
150     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
151     agnostic).
152     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
153     trajectory_to_action_windows =
154     staticmethod(TrackNetRunner.trajectory_to_action_windows)

```

3. user (2026-06-10T06:11:08.370Z)

```

1     """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6     1. Translate the Windows video path to its WSL-visible form (and optionally
7       stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8     2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9     3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10    4. Convert that trajectory into high-motion "action windows" ? the same
11       candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12       pipeline (AI handoff, DB population) is unchanged.
13
14    Nothing here imports TensorFlow; all model code stays in WSL.
15    """
16
17    import os
18    import csv
19    import shlex
20    import logging
21    import subprocess
22    from dataclasses import dataclass
23    from typing import List, Dict, Any, Optional, Tuple
24
25    from backend.pipeline.detectors.base import DetectorRunner
26
27    logger = logging.getLogger(__name__)
28
29
30    @dataclass
31    class TrackNetConfig:

```

```

32     """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33     distro: str = "Ubuntu"
34     repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36     conda_env: str = "TrackNetV4"
37     weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
38     queue_length: int = 5
39     stage_in_wsl: bool = True                       # copy video into WSL fs first
(perf)
40     wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
in WSL
41     timeout_sec: int = 0                            # 0 = no timeout
42
43     @classmethod
44     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
45         tn = (idx_cfg or {}).get("tracknet", {}) or {}
46         return cls(
47             distro=tn.get("wsl_distro", "Ubuntu"),
48             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
49             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
50             conda_env=tn.get("conda_env", "TrackNetV4"),
51             weights_path=tn.get("weights_path", ""),
52             queue_length=int(tn.get("queue_length", 5)),
53             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
54             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
55             timeout_sec=int(tn.get("timeout_sec", 0)),
56         )
57
58
59     def to_wsl_mnt_path(win_path: str) -> str:
60         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
61
62         Already-POSIX paths (starting with / or ~) are returned unchanged so the
63         same helper is safe to call on either kind of path.
64         """
65         p = str(win_path)
66         if p.startswith("/") or p.startswith("~"):
67             return p
68         p = p.replace("\\", "/")
69         if len(p) >= 2 and p[1] == ":":
70             drive = p[0].lower()
71             return f"/mnt/{drive}{p[2:]}"
72         return p
73
74
75     @dataclass
76     class TrajectoryPoint:
77         frame: int
78         visible: bool
79         x: float
80         y: float
81
82
83     class TrackNetRunner(DetectorRunner): # DetectorRunner ABC (finish item 6 / new item 1)
84         """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
85
86         Implements the common DetectorRunner contract so a future Linux-native or
87         alternative trajectory runner can be substituted without touching the hybrids.
88         """
89
90         def __init__(self, cfg: TrackNetConfig):
91             self.cfg = cfg
92

```

```

93 # ----- WSL exec
94
95 def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
96     """Run a bash command string inside the configured WSL distro."""
97     full = f"source {self.cfg.conda_sh} && {inner_cmd}"
98     argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
99     logger.debug("WSL exec: %s", full)
100     return subprocess.run(
101         argv, capture_output=True, text=True,
102         timeout=(self.cfg.timeout_sec or None),
103     )
104
105 def healthcheck(self) -> Tuple[bool, str]:
106     """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
107
108     Returns (ok, message). Cheap to call before a long run.
109     """
110     cmd = (
111         f"conda activate {self.cfg.conda_env} && "
112         f"python -c \"import tensorflow as tf; "
113         f"print('TF', tf.__version__, 'GPUs', "
114         f"len(tf.config.list_physical_devices('GPU'))))\"
115     )
116     try:
117         res = self._wsl_bash(cmd)
118     except FileNotFoundError:
119         return False, "wsl not found ? is this running on Windows with WSL2 installed?"
120     except subprocess.TimeoutExpired:
121         return False, "WSL healthcheck timed out."
122     out = (res.stdout or "").strip()
123     if res.returncode != 0:
124         return False, f"WSL env check failed: {(res.stderr or out).strip()}"
125     return True, out
126
127 # ----- inference
128
129 def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
130     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
131     None.
132
133     ``output_win_dir`` is a Windows directory (under the repo's output/) that
134     WSL writes into via its /mnt view, so the Windows process can read the CSV
135     back with no copy.
136     """
137     if not self.cfg.weights_path:
138         logger.error(
139             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
140             "in config.json to the WSL path of your trained TrackNetV4 weights."
141         )
142         return None
143
144     os.makedirs(output_win_dir, exist_ok=True)
145     # Normalize separators so basename/splitext work when tests (or collector)
146     # pass Windows-style paths on a Linux host.
147     video_win_path = str(video_win_path).replace("\\", "/")
148     base = os.path.basename(video_win_path)
149     stem, ext = os.path.splitext(base)
150
151     # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
152     if ext.lower() not in (".mp4", ".avi"):
153         logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
154         return None
155
156     wsl_out = to_wsl_mnt_path(output_win_dir)

```

```

155         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
156
157         # Resolve the video path WSL will read.
158         if self.cfg.stage_in_wsl:
159             staged = self._stage_video(video_win_path, base)
160             if staged is None:
161                 return None
162             wsl_video = staged
163         else:
164             wsl_video = to_wsl_mnt_path(video_win_path)
165
166         cmd = (
167             f"conda activate {self.cfg.conda_env} && "
168             f"cd {self.cfg.repo_dir}/src && "
169             f"python predict.py "
170             f"--video_path {shlex.quote(wsl_video)} "
171             f"--model_weights {shlex.quote(self.cfg.weights_path)} "
172             f"--output_dir {shlex.quote(wsl_out)} "
173             f"--queue_length {self.cfg.queue_length}"
174         )
175         logger.info("Running TrackNet inference on %s ...", base)
176         try:
177             res = self._wsl_bash(cmd)
178         except subprocess.TimeoutExpired:
179             logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
180             return None
181
182         if res.returncode != 0:
183             logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
184             return None
185
186         if not os.path.exists(expected_csv_win):
187             logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
188             return None
189         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
190         return expected_csv_win
191
192     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
193         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
194
195         Returns the WSL path to the staged copy, or None on failure.
196         """
197         src_mnt = to_wsl_mnt_path(video_win_path)
198         dest = f"{self.cfg.wsl_stage_dir}/{base}"
199         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{shlex.quote(dest)} && echo OK"
200         try:
201             res = self._wsl_bash(cmd)
202         except subprocess.TimeoutExpired:
203             logger.error("Staging video into WSL timed out.")
204             return None
205         if res.returncode != 0 or "OK" not in (res.stdout or ""):
206             logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
207             return None
208         return dest
209
210     # ----- CSV -> windows
211
212     @staticmethod
213     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
214         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""

```

```

215     points: List[TrajectoryPoint] = []
216     with open(csv_win_path, newline="") as f:
217         reader = csv.DictReader(f)
218         for row in reader:
219             try:
220                 points.append(TrajectoryPoint(
221                     frame=int(row["Frame"]),
222                     visible=int(row["Visibility"]) == 1,
223                     x=float(row["X"]),
224                     y=float(row["Y"]),
225                 ))
226             except (KeyError, ValueError):
227                 continue
228     points.sort(key=lambda p: p.frame)
229     return points
230
231 @staticmethod
232 def trajectory_to_action_windows(
233     points: List[TrajectoryPoint],
234     fps: float,
235     frame_width: float,
236     chunk_start: float = 0.0,
237     velocity_thresh: float = 0.02,
238     merge_gap: float = 2.0,
239     min_window_duration: float = 2.0,
240     chunk_index: Optional[int] = None,
241 ) -> List[Dict[str, Any]]:
242     """Convert a shuttle trajectory into candidate rally windows.
243
244     A frame is "active" when the shuttle is visible AND its inter-frame
245     displacement (normalised by frame width, per second) exceeds
246     ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
247     ``merge_gap`` seconds of each other are grouped into one window; short
248     windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
249     shape feeding the AI-handoff phase is identical.
250     """
251     if fps <= 0:
252         fps = 30.0
253     if frame_width <= 0:
254         frame_width = 1280.0
255
256     # Compute per-frame normalised velocity from the last *visible* point.
257     active = [] # (frame_idx, velocity)
258     prev: Optional[TrajectoryPoint] = None
259     for p in points:
260         if not p.visible:
261             prev = None # break the trajectory; absent shuttle = not in flight
262             continue
263         if prev is not None:
264             dframes = p.frame - prev.frame
265             if dframes > 0:
266                 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
267                 velocity = (dist / frame_width) * (fps / dframes)
268                 if velocity > velocity_thresh:
269                     active.append((p.frame, velocity))
270         prev = p
271
272     if not active:
273         return []
274
275     max_gap_frames = int(merge_gap * fps)
276
277     def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
278         vels = [v for _, v in group]
279         return {

```



```

280         "start": group[0][0] / fps + chunk_start,
281         "end": group[-1][0] / fps + chunk_start,
282         "active_frame_count": len(group),
283         "peak_velocity": max(vels),
284         "mean_velocity": sum(vels) / len(vels),
285         "track_id_count": 1, # single shuttle; kept for window-shape parity
286         "chunk_index": chunk_index,
287     }
288
289     windows = []
290     group = [active[0]]
291     for af in active[1:]:
292         if af[0] - group[-1][0] <= max_gap_frames:
293             group.append(af)
294         else:
295             windows.append(_finalize(group))
296             group = [af]
297     windows.append(_finalize(group))
298
299     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
300

```

4. user (2026-06-10T06:11:16.677Z)

```

config.json:54:         "timeout_sec": 0,
docs\CODE_MAP.md:121:[Omitted long matching line]
docs\CODE_MAP.md:255:- **`timeout_sec=0` = no timeout:** a hung WSL/GPU call blocks forever.
backend\config\models.py:42:     timeout_sec: int = 0
docs\TRACKNET_WSL_SETUP.md:66: "timeout_sec": 0,           // 0 = no timeout
backend\pipeline\detectors\tracknet_runner.py:41:     timeout_sec: int = 0
# 0 = no timeout
backend\pipeline\detectors\tracknet_runner.py:55:
timeout_sec=int(tn.get("timeout_sec", 0)),
backend\pipeline\detectors\tracknet_runner.py:102:         timeout=(self.cfg.timeout_sec or
None),
backend\pipeline\detectors\tracknet_runner.py:179:         logger.error("TrackNet inference
timed out after %ss.", self.cfg.timeout_sec)
backend\pipeline\detectors\wasb_runner.py:40:     timeout_sec: int = 0 # 0 = no timeout
backend\pipeline\detectors\wasb_runner.py:54:         timeout_sec=int(w.get("timeout_sec",
0)),
backend\pipeline\detectors\wasb_runner.py:67:
timeout=(self.cfg.timeout_sec or None))
backend\pipeline\detectors\wasb_runner.py:137:         logger.error("WASB inference timed out
after %ss.", self.cfg.timeout_sec)

```

5. user (2026-06-10T06:11:18.058Z)

```

108-- **SQLite thread-safety:** verify `check_same_thread`/connection-per-thread;
109-  `_get_connection` already re-issues `PRAGMA foreign_keys=ON` per connection ?
110-  confirm it's safe under the executor.
111:- Cancellation/timeout and crash recovery (a job in RUNNING after a restart).
112-- In-process store is not horizontally scalable ? acceptable for the current
113-  single-box self-hosted target; call it out so it isn't mistaken for production-HA.
114-

```

6. assistant (2026-06-10T06:11:25.100Z)

The exact behavior is documented in CODE_MAP.md line 255. Let me see the context.

7. user (2026-06-10T06:11:26.521Z)

```

240     - **`output_dir`:** a real field on `OutputConfig` (default `"output"`); the CLI
`--output-dir` overrides it on the typed model in `main()`, and `compile_highlights` reads the
same `global_config.output.output_dir` ? honored end-to-end.

```

```

241     - **Gemini free-tier / API-key dead path:** hybrids + gemini return ``[]`` (not error) if
  `{PROVIDER}_API_KEY` missing OR WSL healthcheck fails ? a silent no-op. `analyze_video` also
  returns ``[]`` on real failure -> empty != "no rallies". The reporting
  `silent_provider_noop`/`ai_empty_vs_no_rallies` rules exist precisely to disambiguate this in
  the report.
242     - **Reporting never breaks a run:** `emit_indexer_report` swallows all exceptions
  (designed for `main.py` `finally:`) and is emitted even on validation failure. `obsreport` is a
  SOFT dep ? absent -> no-op.
243     - **spec.yaml <-> harvest.py coupling:** rule `path` strings dotted-navigate harvest's
  stage/metric/detail names; rename a detail in harvest.py and the matching rule silently stops
  firing. Add rules in spec.yaml, not code.
244     - **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` / runner
  fallbacks (30.0, 1280.0) silently mis-scale velocity thresholds; calibrate_wasb CLI bakes
  `59.94/1920`. The report's `fps_fallback_used` flag surfaces this.
245     - **WSL `/mnt` can't see Google Drive / network mounts;** stage video into native WSL fs
  (`~/clips`) ? `/mnt/c` reads are slow and Drive-backed paths invisible.
246     - **ffmpeg NOT in WSL** by design: all ffmpeg/ffprobe run Windows-side (must be on
  PATH); WSL only runs the GPU model. `extract_frames` uses cv2 PNG (slow) ? prefer host ffmpeg +
  `--frames_dir`. ? Don't import `wasb_infer` from Windows code (it imports torch + WASB repo
  modules).
247     - **GPU float nondeterminism:** detector pass is checkpointed/resumable but not bit-
  reproducible; the CPU tracker re-run over the ordered cache IS deterministic.
248     - **`output/` is gitignored:** DB, baselines, cache, reels live there. Predictions MUST
  exist in `output/<db_name>` before any eval/regression.
249     - **Validator ordering coupling:** exec order is Format -> ResolutionFPS ->
  PTSContinuity -> SceneCut -> Blur -> Relevance; `resolution_fps_check` hard-fails unless
  `format_check` ran earlier (shared `context['ffprobe_meta']`). PTS 10.0s gap + scene-cut 0.6
  correlation are NOT config-overridable.
250     - **`add_video` INSERT OR REPLACE + FK CASCADE** -> re-ingesting a `video_id` cascade-
  deletes its segments. Use `clear_video_data` deliberately. `_get_connection` MUST re-issue
  `PRAGMA foreign_keys=ON` per connection.
251     - **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-paste; fixes to
  P3/P4 must land in both. ? subtle drift: wasb's shot_count `int(None)`-via-TypeError fallback vs
  tracknet's `is None` check.
252     - **Two "annotations" modules:** `backend.eval.annotations` (GT CSV loader) vs
  `backend.annotations` (provider registry). regression.py imports the latter.
253     - **`calibrate_wasb.{load_golden_gts, make_predict_fn, BASELINE}` is a de-facto shared
  API** for calibrate_local, export_wasb_segments, gemini_refine, e1_probe ? changing its golden-
  CSV schema or `BASELINE` ripples widely. `gemini_refine.merge_overlaps` reused by
  `distill_local`; `harness.get_predictions` reused by `regression.regress`.
254     - **Hardcoded tuned constants:** `calibrate_wasb.BASELINE`, `export.TUNED`,
  slicer/compiler padding ? duplicated by copy, not import; re-tuning won't propagate.
255     - **`timeout_sec=0` = no timeout:** a hung WSL/GPU call blocks forever.
256     - **Stitch concat assumes identical encode:** per-clip re-encode then `-c copy` concat;
  if probe fails (0.0) no boundary validation.
257     - **Audio NOT in the live pipeline:** `pipeline/audio/hit_detector` +
  `eval/audio/e1_probe` are a diagnostic probe (ADR-007); recall 100% but discrimination ~2.2x ->
  not adopted for fusion.
258
259     ---
260
261     ## 7. RAMP PATHS (to do X -> open Y::symbol)
262     | Task | Open |
263     |---|---|
264     | Change a config default / add a config field | `@backend/config/models.py` (single
  source of truth) -> regenerate via `setup.py`/`save_default_config`; ? config.json may override
  (default_segmenter) |
265     | Read config in new code | `from backend.config import load_config, AppConfig`
  (`@backend/config/__init__.py`); attribute access, NOT raw `json.load` |
266     | Tune candidate windowing (velocity/merge/min-dur) |
  `@backend/pipeline/detectors/tracknet_runner.py::trajectory_to_action_windows`; cfg `config.json
  indexing.{wasb,tracknet}.velocity_threshold`, `dead_time_merge_gap`,
  `min_action_window_duration` |
267     | Add a segmenter | subclass `@backend/pipeline/segmenters/base.py::BasePlaySegmenter`;
  `segmenter_registry.register(...)` at module bottom; import in

```

```
`@backend/pipeline/segmenters/__init__.py`; if AI-based add to
`@backend/reporting/harvest.py:::_AI_SEGMENTERS` |
268 | Change stitching padding | `config.json stitching.{begin_padding,end_padding}`; impl
`@backend/pipeline/stitcher/compiler.py::VideoCompiler.compile_highlights`; mirror
`@backend/tools/rally_slicer.py::BEGIN_PADDING/END_PADDING` |
269 | Add a sport | subclass `@backend/sports/base.py::SportProfile`;
`sport_registry.register(...)` in `@backend/sports/__init__.py`; define prompt+HSV in new file |
```

8. user (2026-06-10T06:11:27.352Z)

```
115
116     ### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)
117     - `@backend/pipeline/detectors/base.py` ? DetectorRunner ABC (healthcheck() ->
Tuple[bool,str] never-raises; run_predict(...) -> Optional[str] CSV path). Contract deliberately
matches the pre-existing tuple returns and call sites in the hybrids. Rich module docstring
includes "how to add a new runner", Linux-native path notes (the de-risk goal), and guidance on
threading health into reports. Re-exported from `detectors/__init__.py`. This finishes the WSL
de-risk seam (low-regret 6 / review item 1).
118     - `@backend/pipeline/detectors/tracknet_runner.py`
119     - `::TrackNetConfig` (+`.from_indexing_cfg`; ? key drift `wsl_distro` JSON -> `distro`
field), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
120     - `::to_wsl_mnt_path` (`C:\x`->`/mnt/c/x`; ? POSIX/~` passthrough, UNC unhandled),
`::TrajectoryPoint` (frame,visible,x,y).
121     - `**::parse_trajectory_csv` @staticmethod** + `**::trajectory_to_action_windows`
@staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path`
defaults `` -> `run_predict` returns None; predict.py only `.mp4/.avi` + hard-names
`<stem>_predict.csv`; `visible==False` resets prev (occlusion gap breaks track); `fps<=0->30`,
`frame_width<=0->1280` silent; `track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout
(hangs forever). `::stage_video` "OK" sentinel.
122     - `@backend/pipeline/detectors/wasb_runner.py::WasbRunner` (inherits DetectorRunner)
(+`::WasbConfig`, from `indexing.wasb`; ? `weights_path` HAS a real default
`wasb_badminton_best.pth.tar` ? no required guard) ? stages `wasb_infer.py` (`::INFER_WIN`) +
video into WSL, runs in `wasb` env. `::parse_trajectory_csv`/`::trajectory_to_action_windows` =
`staticmethod(TrackNetRunner.*)` (explicit rebind = the plugin-equivalence point). Output hard-
named `<stem>_wasb.csv`. ? `wasb_infer.py` re-staged each run (editing Windows copy inert until
staged); `wsl_bash` duplicated (drift risk). Both runners now satisfy the DetectorRunner ABC
for future swappability.
123     - `@backend/pipeline/detectors/wasb_infer.py` (WSL) ? `::run` core. ? imports WASB-SBDT
repo modules + torch ? NOT importable on Windows. Reboot-resumable detector cache:
`::DET_FILE='det_raw.jsonl'`, `::MANIFEST_FILE='manifest.jsonl'`, `::CACHE_VERSION=1`,
`::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line index`, rewrites to
heal crash-truncated tail), `::manifest_compatible` (? `frames_dir` abspath'd but `weights`
compared raw), `::atomic_write_text`/`atomic_write_trajectory`, `::dets_to_tracker` (xy MUST be
np.array), `::build_cfg` (`runner.gpus=[0]` hardcoded), `::extract_frames` (cv2-PNG SLOW),
`::default_cache_dir` (`<stem>_wasbcache` next to out). ? GPU pass resumable; CPU tracker
STATEFUL -> always fully re-run; `--fresh` ignores cache.
124     - `@backend/pipeline/detectors/rally_gate.py` ? Gate A net-crossing post-filter (pure,
no I/O/model). `::estimate_play_axis` (larger-spread axis; ? ties -> 'x'),
`::count_net_crossings` (sign changes vs median net; deadband jitter zone),
`::gate_windows`/`::apply_gate` (`min_crossings=2` default; kills single-toss service feeds).
`::GatedWindow` (start,end,crossings,visible_points,kept). ? window contract here is
`(start,end)` tuples NOT the rich window dict ? caller must adapt. Consumed by calibrate_local /
distill_local / export_wasb_segments.
125
126     ### 2.3 Eval & calibration (all pure core; I/O in CLI drivers)
```

9. user (2026-06-10T06:11:42.112Z)

```
backend\api\static\app.js:87:         const response = await fetch("/api/process", {
backend\main.py:91:@app.post("/api/process")
backend\main.py:92:def process_video(req: ProcessRequest):
backend\pipeline\segmenters\base.py:24:     def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
backend\pipeline\segmenters\gemini.py:25:     def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
```

```

backend\pipeline\segmenters\gemini.py:137:         def process_single_chunk(ci: int,
chunk_start: float) -> Tuple[str, list, dict]:
backend\pipeline\segmenters\gemini.py:193:         with
concurrent.futures.ThreadPoolExecutor(max_workers=5) as executor:
backend\pipeline\segmenters\motion.py:22:     def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend\pipeline\segmenters\tracknet_hybrid.py:59:     def process_video(self, video_id: str,
video_path: str, player_pool: List[str] = None,
backend\pipeline\segmenters\tracknet_hybrid.py:180:         def process_candidate_window(wi: int,
window: Dict[str, Any]) -> List[dict]:
backend\pipeline\segmenters\tracknet_hybrid.py:206:         with
concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
backend\pipeline\segmenters\wasb_hybrid.py:56:     def process_video(self, video_id: str,
video_path: str, player_pool: List[str] = None,
backend\pipeline\segmenters\wasb_hybrid.py:160:         def process_candidate_window(wi: int,
window: Dict[str, Any]) -> List[dict]:
backend\pipeline\segmenters\wasb_hybrid.py:186:         with
concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
backend\eval\gemini_refine.py:121:     with
concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
backend\eval\gemini_refine.py:186:     with
concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
backend\pipeline\segmenters\yolo_hybrid.py:263:     def process_video(self, video_id: str,
video_path: str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend\pipeline\segmenters\yolo_hybrid.py:351:         extract_executor =
concurrent.futures.ThreadPoolExecutor(max_workers=prefetch_workers)
backend\pipeline\segmenters\yolo_hybrid.py:485:         def process_candidate_window(wi: int,
window: Dict[str, Any]) -> List[dict]:
backend\pipeline\segmenters\yolo_hybrid.py:518:         with
concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:

```

10. user (2026-06-10T06:11:43.020Z)

```

60     that a video used to train the learned segmenter
61     (`backend/eval/training.py::build_training_set` ? `distill_local`) is absent from the
62     eval/regression set. Leakage would silently inflate the very metrics the product
63     story rests on (? in BACKLOG).
64
65     ### Approach
66     - **Split unit = match, never clip** (per `GOLDEN_SET_PHASE1_VIDEOS.md`: split by
67       match, hold ~? of each sport as eval).
68     - Add a `split: "train" | "eval"` field to the collector's `curate export`
69       manifest (cross-repo contract, ADR-002 lineage) ? or a separate eval manifest.
70     - New `backend/eval/splits.py`:
71     - `load_split(manifest) -> (train_ids, eval_ids)` keyed by **match id** (not just
72       file MD5 ? re-encoded duplicates share a match but differ in
73       `compute_video_id`, so group at match level).
74     - `assert_disjoint(train_ids, eval_ids)` ? raise on any overlap.
75     - Wire the guard into the training entrypoint (`distill_local`/`training`) and into
76       `run_regression.py` so the baseline is computed strictly on the eval split.
77
78     ### Risks
79     - Requires a collector-side manifest change (coordinate the cross-repo contract).
80     - Small corpus ? tiny held-out set ? high metric variance; the guardrail is correct
81       but the numbers will be noisy until the golden set grows.
82     - MD5-only dedup is insufficient (re-encodes evade it) ? must track match identity.
83
84     ### Verification
85     - Unit test: overlapping train/eval ids ? `assert_disjoint` raises.
86     - Integration: a deliberately-leaked manifest fails the training entrypoint.
87
88     ---
89
90     ## 3. Async job model (debt #16)
91

```

```

92     ### Problem
93     `POST /api/process` runs validate ? segment ? report inline, blocking the
94     request for the full duration of a long video. Unworkable as a hosted/product API.
95
96     ### Approach (minimal first slice ? single self-hosted box)
97     - `POST /api/process` returns 202 + `job_id`; add `GET /api/jobs/{job_id}`
98       returning `{status, progress, result, reports}`.
99     - Run the pipeline on a background worker ? start with a `ThreadPoolExecutor`
100      (segmenters already use threads internally for AI handoff, so this fits); leave a
101      real queue (RQ/Celery) for later if multi-process is needed.
102     - Persist job state in the existing SQLite DB via the migration ladder
103      (`backend/infrastructure/database.py::_init_db`, bump `PRAGMA user_version` 1 ? 2,
104      add a `jobs` table). The observability report becomes the job's artifact.
105     - Update the static UI to poll `GET /api/jobs/{id}` instead of blocking.
106
107     ### Risks
108     - SQLite thread-safety: verify `check_same_thread`/connection-per-thread;
109       `_get_connection` already re-issues `PRAGMA foreign_keys=ON` per connection ?
110       confirm it's safe under the executor.
111     - Cancellation/timeout and crash recovery (a job in RUNNING after a restart).
112     - In-process store is not horizontally scalable ? acceptable for the current
113       single-box self-hosted target; call it out so it isn't mistaken for production-HA.
114

```

11. assistant (2026-06-10T06:12:01.789Z)

One more check ? whether the default segmenter used by /api/process even invokes the WSL runners.

12. user (2026-06-10T06:12:03.390Z)

```

README.md:74:This outputs a validation summary and indexes all rallies directly into the local
SQLite database, using whichever segmenter is set as `default_segmenter` in `config.json`
(`yolo_hybrid` by default).
README.md:124:     "default_segmenter": "yolo_hybrid",
README.md:155:*   `indexing.default_segmenter` ? `yolo_hybrid` | `gemini` | `motion`.
README.md:215:   *Or set `"default_segmenter": "yolo_hybrid"` (or `"gemini"`/`"motion"`) under
`"indexing"` in `config.json`.
README.md:219:[Omitted long matching line]
README.md:237:     "default_segmenter": "gemini",
config.json:26:     "default_segmenter": "gemini",
docs\CODE_MAP.md:29: -> seg_name = req.segmenter or global_config.indexing.default_segmenter
(fallback 'motion')
docs\CODE_MAP.md:86: - ? `--segmenter` argparse `choices` frozen at build time from
`segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI
default_segmenter fallback `motion`.
docs\CODE_MAP.md:96:[Omitted long matching line]
docs\CODE_MAP.md:102:- `@config.json` ? on-disk config; mirrors `AppConfig`. ?
`indexing.default_segmenter='gemini'` diverges from model default `yolo_hybrid` ? file
wins at runtime.
docs\CODE_MAP.md:177:[Omitted long matching line]
docs\CODE_MAP.md:238:- Segmenter default: single source of truth = model
`IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships
`gemini`). run*.py read the model value (no literals); `main.py` keeps the defensive `motion`
getattr fallback noted above.
docs\CODE_MAP.md:264:| Change a config default / add a config field |
`@backend/config/models.py` (single source of truth) -> regenerate via
`setup.py`/`save_default_config`; ? config.json may override (default_segmenter) |
backend\config\models.py:47:     default_segmenter: str = "yolo_hybrid"

```

[Showing results with pagination = limit: 15]

13. user (2026-06-10T06:12:24.639Z)

Structured output provided successfully

14. assistant (2026-06-10T06:12:29.327Z)

Verification complete.